

Faster single-source shortest paths with negative real weights via proper hop distance

Combinatorial Optimization, Spring 2026

Vasileios Gkolosi, Konstantinos Chousos

National & Kapodistrian University of Athens

May 8, 2026

Introduction

Introduction i

Definition 1 (Single-Source Shortest Path Problem)

Let $G = (V, E)$ be a graph. For a source vertex $s \in V$ and a weight function $w : E \rightarrow \mathbb{R}$, find the **shortest path** (i.e. min distance) to all other vertices $t \in V \setminus s$.

Introduction ii

Definition 2 (Distance)

The distance between two vertices is defined as

$$d(s, t) = \min\{c(P) \mid P \text{ is an } s - t \text{ path}\}.$$

Definition 3 (Path cost)

Let $P = (v_1, \dots, v_n) \in \cup_{k \in \mathbb{N}} V^k$ where $\{v_i, v_{i+1}\} \in E$. The cost of the path is defined as

$$c(P) = \sum_{i=1}^{n-1} w(v_i, v_{i+1}).$$

Introduction iii

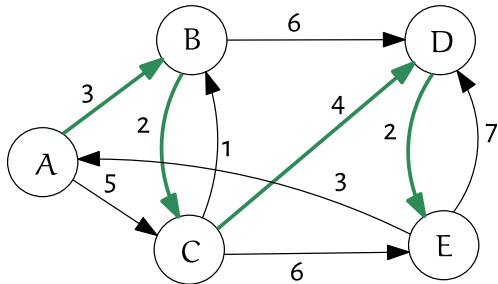


Figure 1: An example graph, with its SSSP solution for the source A in green.

Introduction iv

- SSSP is well defined *if and only if* no negative cycle is traversable from s .
- Negative cycles create negative infinite distances.

Introduction v

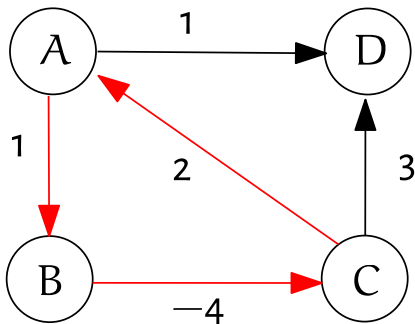


Figure 2: A graph containing a negative cycle (denoted in red).

Introduction vi

- In this case, the problem is not well defined.
- Thus, we also want to know if such a cycle exists.

SSSP Algorithms

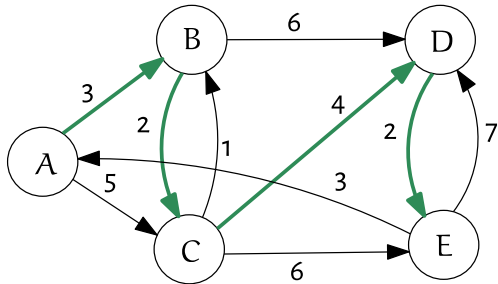
Dijkstra's algorithm i

Theorem 4 (Dijkstra, 1959)

Let $G = (V, E)$ be a graph, $n = |V|$, $m = |E|$ and w a nonnegative weight function. There exists an algorithm that solves SSSP in time

$$O(m + n \log n).$$

Dijkstra's algorithm ii



Dijkstra's algorithm iii

Problem: Dijkstra's algorithm **does not work** for negative weights.

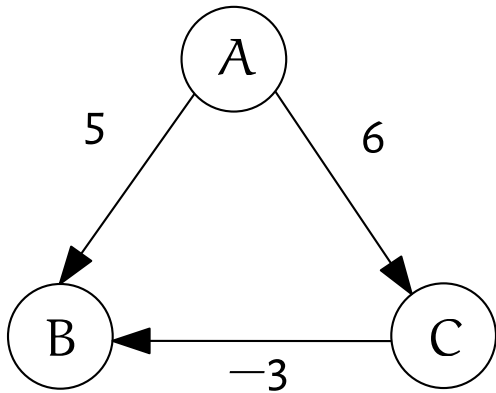


Figure 3: A graph with a negative edge.

Bellman-Ford algorithm i

Theorem 5 (Shimbel, Bellman, Ford, Moore 1955–59)

Let $G = (V, E)$ be a graph, $n = |V|$, $m = |E|$ and $w : E \rightarrow \mathbb{R}$ a weight function. There exists an algorithm that solves SSSP in time

$$O(mn).$$

Bellman-Ford algorithm

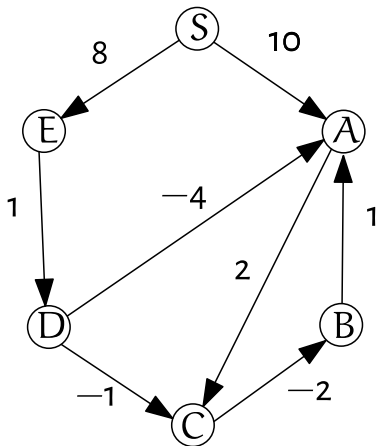


Figure 4: A graph with negative edges.

Table 1: The distances from source node S for the graph of fig. 4, computed with the BF algorithm.

S	A	B	C	D	E
0	5	5	7	9	8

$$(G_{w(e) \in \mathbb{R}} \preceq G_{w'(e) \in \mathbb{R}^+})$$

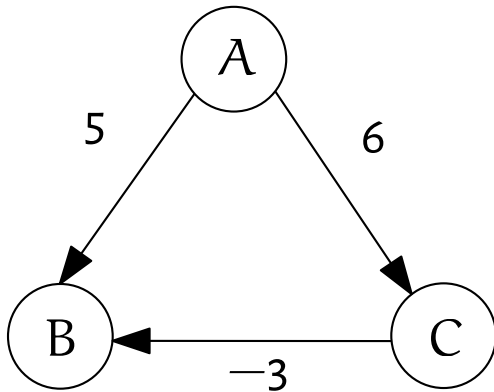
Goal: Reduce a graph with real weights to a graph with nonnegative weights.

Idea: Let a new weight function $w' = w + C$, where

$$C = -\min_{e \in E} \{w(e)\}.$$

$$(G_{w(e) \in \mathbb{R}} \preceq G_{w'(e) \in \mathbb{R}^+}) \text{ ii}$$

Problem: w' does not conserve the shortest paths.



$$(G_{w(e) \in \mathbb{R}} \leq G_{w'(e) \in \mathbb{R}^+}) \text{ iii}$$

Alternative idea: Add a telescoping term instead:

$$w_\varphi(u, v) := w(u, v) + \varphi(u) - \varphi(v),$$

where $\varphi : V(G) \rightarrow \mathbb{R}$.

$$(G_{w(e) \in \mathbb{R}} \leq G_{w'(e) \in \mathbb{R}^+}) \text{ iv}$$

For any path P of length k :

$$\sum_{n=0}^{k-1} w_{\varphi}(v_n, v_{n+1}) = \sum_{n=0}^{k-1} w(v_n, v_{n+1}) + \varphi(v_0) - \varphi(v_k)$$

Thus, the cost of a path $P = (v_1, \dots, v_n)$ becomes

$$c_{w_{\varphi}}(P) = c_w(P) + \varphi(v_1) - \varphi(v_n).$$

Result: Shortest paths are **conserved**.

Potential functions i

Such $\varphi : V(G) \rightarrow \mathbb{R}$ functions are called **potential/price functions**. If $w_\varphi(u, v) \geq 0 \forall \{u, v\} \in E(G)$ where $w(u, v) \geq 0$, φ is called a *valid* potential function. I.e. it does not introduce new negative edges.

Note

If φ_1, φ_2 are potentials, so are $\varphi_1 + \varphi_2, -\varphi_1, \max\{\varphi_1, \varphi_2\}$.

Johnson's potentials i

1. Let G be a graph. Add new *super source* vertex s' with zero-weight edges to all vertices. Let G' be the resulting graph. Note that the triangle inequality holds:

$$d(s', u) + w(u, v) \geq d(s', v).$$

Johnson's potentials ii

2. Set $\varphi(v) := d_{G'}(s', v)$.

$$d(s', u) + w(u, v) \geq d(s', v) \quad \implies$$

$$\varphi(u) + w(u, v) \geq \varphi(v) \quad \implies$$

$$w(u, v) + \varphi(u) - \varphi(v) \geq 0.$$

We have now obtained a **valid** potential function such that $w_\varphi(e) \geq 0, e \in E^+(G)$.

Johnson's potentials iii

- A more fitting potential, is $\varphi(t) = d(V, t), \forall t \in V$, which neutralizes all negative edges when it is definable.
- More generally, the potential $\varphi(t) = d(A, t) \forall t \in V$, where $A \supseteq S \subseteq N$ neutralizes all negative edges corresponding to the participating vertices in S .

Johnson's algorithm i

Theorem 6 (Johnson, 1977)

Let $G = (V, E)$ be a graph, $n = |V|$, $m = |E|$ and $w : E \rightarrow \mathbb{R}$ a weight function. There exists an algorithm that solves the SSSP problem in time

$$O(n \log n + mn).$$

Johnson's algorithm ii

Let input graph G .

1. Add super source s' and zero-weight edges.
2. Compute $\varphi(v)$, $v \in V(G)$ by computing $w(s', v)$ through BF.
 - 2.1 Terminate if negative cycle is detected.
3. Compute Dijkstra(v), $v \in V(G)$ with the new weight function w_φ .

Problem: still need initial Bellman-Ford run to compute $\varphi(u)$.

Bellman-Ford-Dijkstra algorithm

Theorem 7

Let $G = (V, E)$ be a graph, $n = |V|$, $m = |E|$, and $w : E \rightarrow \mathbb{R}$ a weight function. There exists an algorithm that solves h -limited¹ SSSP in time

$$O(h \cdot (n \log n + m)).$$

¹For any vertex reachable from the source, there exists a shortest path to it with at most h negative edges.

Preliminaries

Union Bound & Chernoff bound

Theorem 8 (Union bound)

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space. Then

$$A_1, \dots, A_n \in \mathcal{F} \implies \mathbb{P} \left(\bigcup_{i \leq n} A_i \right) \leq \sum_{i \leq n} \mathbb{P}(A_i).$$

Theorem 9 (Chernoff bound for binomials)

Let $X \sim \text{Bin}(n, p)$. Then $\mu = \mathbb{E}[X] = np$ and

$$\begin{aligned} \mathbb{P} \left[X \leq \frac{\mu}{2} \right] &\leq e^{-\frac{\mu}{8}} \\ \mathbb{P} \left[X > \frac{3\mu}{2} \right] &\leq e^{-\frac{\mu}{12}} \end{aligned}$$

Graph Preprocessing i

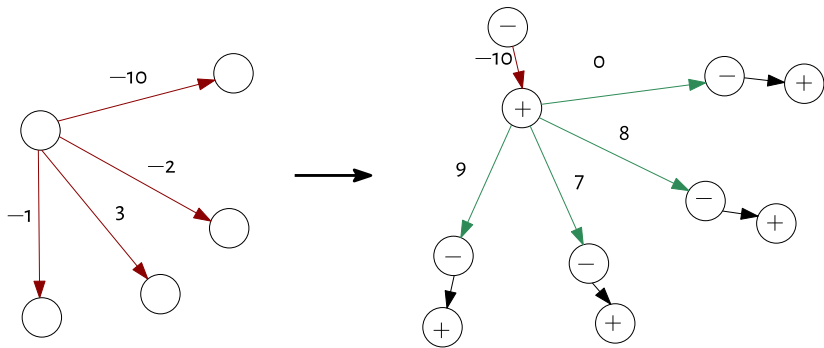


Figure 5: Example instance of the graph preprocessing algorithm.

Graph Preprocessing ii

For each vertex v , let $\mu_v = \min\{w(u, x) \mid (v, x) \in E(G)\}$ be the minimum length of the edges *leaving* v .

We construct G' as follows:

1. Replace each vertex v with two vertices v^- , v^+ and edge (v^-, v^+) of length μ_v .
2. Replace each edge (u, v) with the edge (u^+, v^-) of length $w(u, v) - \mu_u$.

Hop distance i

Definition 10 (h-hop path)

A path with at most h negative edges.

Definition 11 (h-hop distance)

Let $s, t \in V$. We denote $d^r(s, t)$ as the minimum length over all h -hop walks from s to t .

Hop distance ii

- If $v \in V$ has a negative outgoing edge e ($w(e) < 0$), then we denote v a *negative* vertex.
- Let $N \subseteq V$ be the negative vertices in G .

Hop distance iii

Hop distances obey the following recurrence:

$$d^{h+1}(s, t) = \min \left\{ d^h(s, t), \min_{u \in N} d^h(s, u) + d^1(u, t) \right\},$$

as well as the triangle inequality:

$$d^{h+1}(s, t) \leq d^h(s, u) + d^1(u, t).$$

If $d^{h+1}(s, v) = d^h(s, v) \forall v \in N$, then

$$d(s, v) = d^{h+1}(s, v) \forall v \in N.$$

Hop distance iv

Lemma 12

For any $S \subseteq V$, $\varphi_1(v) = d^h(S, v)$ and $\varphi_2(v) = -d^h(v, S)$ are both *valid*.

Main result

Main result

Theorem 13 (Huang et al., 2024)

There is a (Las Vegas) randomized algorithm solving the SSSP problem for real-weighted graphs in time

$$O\left(mn^{4/5} \log^{2/5} n + n^{9/5} \log^{7/5} n\right),$$

with high probability.

Sketch i

1. Iteratively:
 - 1.1 Find set of negative vertices S
 - 1.2 Work on its induced subgraph G_S
 - 1.3 Compute valid potential function φ in G_S
 - 1.4 Neutralize all negative edges in G_S
 - 1.5 Set $G \leftarrow G_\varphi$
2. End up with only nonnegative weights
3. Run Dijkstra on the resulting graph

Sketch ii

- Remote supersets of negative vertices $\Rightarrow$ local reweighting
- Problem: such supersets are rare to find
- Solution: “massage” the graph

Hop Reduction

Hop reduction i

- Johnson solves SSSP with Bellman-Ford-Dijkstra, thus neutralizes negative edges with a runtime of

$$O(k \cdot (n \log n + m))$$

- Bottleneck: $O(m + n \log n)$ per edge
- Idea: From arbitrary graph G , create auxiliary graph H s.t.
 1. $V(G) \subseteq V(H)$
 2. $d_G(u, v) \leq d_H^{\lceil h/r \rceil}(u, v) \leq d_G^h(u, v)$, i.e. h -hop path in $G \rightarrow h/r$ -hop path in H .
- H is called an r -hop reduction of G

Hop reduction ii

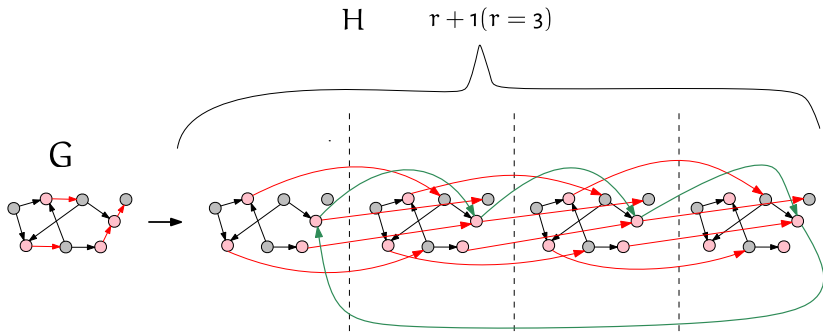


Figure 6: A rudimentary construction of auxiliary graph H .

Hop reduction iii

Rudimentary construction of H :

1. Create $r + 1$ layered copies of the graph $(V(G), E^+(G))$
2. Add the negative edges between layers If $(u, v) \in N$, add (u_i, v_{i+1}) to E_H for $0 \leq i < r$
3. Add zero-length edges (v_i, v_{i+1}) for all $v \in V, 0 \leq i < r$ and (v_r, v_0)

$$n_H = O(rn), \quad m_H = O(rm).$$

Hop reduction iv

Construction of w_H :

- Johnson's reweighting limited to i hops using BFD for all $i \leq r$

$$O(r(m + n \log n)).$$

- $w_H(u_i, v_j) = w(u, v) + d_G^i(V, u) - d_G^j(V, v)$
- For each $(u, v) \in E$, it follows that $w_H(u_i, v_{i+1}) \geq 0$

Hop reduction v

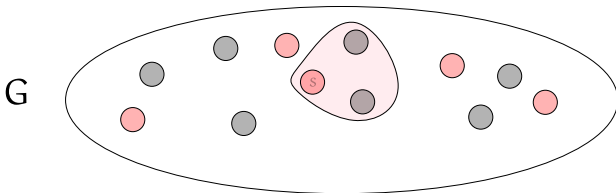
- We can compute $\lceil h/r \rceil$ -limited SSSP on H instead of G with cost of $O(r(k/r) \cdot (n \log n + m))$
- Problem: Runtime remains the same
- Goal: Improve runtime by leveraging an **r-remote set**

Reach

Definition 14 (Negative r-hop reach)

Let $G = (V, E, w)$ and $v \in V$. The set of vertices that can be reached by a negative-weight r-hop path from v is called its **negative r-hop reach**. More generally, we define the negative r-hop reach of a set X as

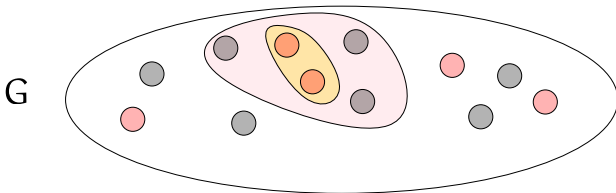
$$R^r(X) = \{v \in V \mid d^r(X, v) < 0\}.$$



Remoteness

Definition 15 (r -remoteness)

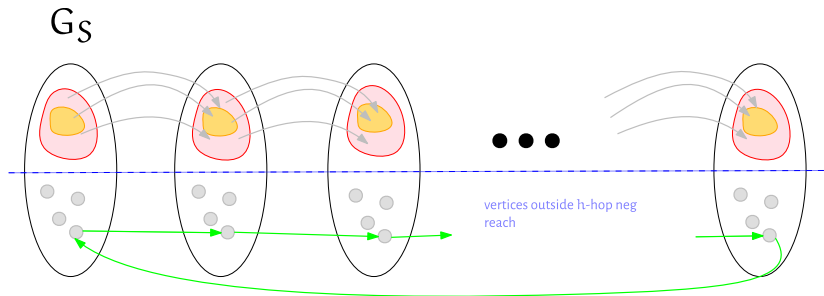
Let $G = (V, E, w)$, $n = |V|$ and $X \subseteq V$ be a subset of negative vertices. X is an **r -remote set** if it has negative r -hop reach of size at most n/r —i.e. $|R^r(X)| \leq n/r$.



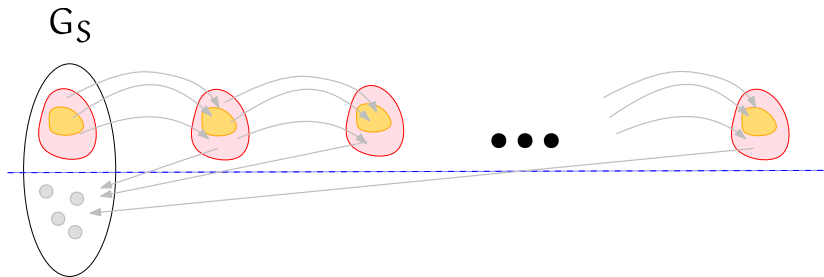
Hop reduction (cont.) i

- Let N be r -remote
- Let $v \in V \setminus R$ be a vertex outside of the r -remote subgraph
 - Then $d_G^i(V, v) = 0, i \leq r$
- Each copy is weighted identically, layered versions can be compressed

Hop reduction (cont.) ii



Hop reduction (cont.) iii



Hop reduction (cont.) iv

Construction of H for G with r -remote set:

1. Create r layered copies of the **r -remote subgraph**
2. Add the negative edges between layers If $(u, v) \in E$, add (u_i, v_{i+1}) to E_H for $0 \leq i < r$
3. Add edges (v_i, v_0) for all $v \in R, i$

$$n_H = r \frac{n}{r} + n = O(n), \quad m_H = O\left(r \frac{n}{r} \frac{m}{n} + m\right) = O(m).$$

Hop reduction (cont.) v

Lemma 16 (Fineman, 2024, lem. 3.3)

Consider a (sub)graph $G = (V, E^+ \cup N, w)$, let $k = |N|$ and $m = |E^+ \cup N|$. Suppose that $w(e) \geq 0$ for all $e \in E^+$ and that N is r -remote. Then there exists an $O((r + k/r)(n \log n + m))$ -time deterministic algorithm that either:

1. correctly determines that the graph contains a negative-weight cycle,
or
2. computes a valid reweighting that eliminates all edges N .

Betweenness

Definition 17 (Betweenness)

Let, $r \in \mathbb{N}$, $s, t \in V$. The r -betweenness of is defined as the number of vertices $u : d^r(s, v) + d^r(v, t) < 0$.

Betweenness

Definition 17 (Betweenness)

Let, $r \in \mathbb{N}$, $s, t \in V$. The r -betweenness of v is defined as the number of vertices u : $d^r(s, v) + d^r(v, t) < r$.

Lemma 18 (Huang et al., 2024, lem 2.2)

There is an $O(rbm \log n + rbn \log^2 n + b^2 n \log^2 n)$ time randomized algorithm that returns either a set of valid potentials φ or a negative cycle. With high probability, all pairs (s, t) have r -betweenness at most n/b after reweighing.

Sandwich

Definition 19 (Weak h -hop negative sandwich)

For a parameter h , a weak h -hop negative sandwich is a triple

$$(s, U, t) : d^h(s, u) + d^h(u, t) \leq 0, \forall u \in U.$$

Betweenness + Sandwich $\rightarrow$ Remoteness

Theorem 20 (Huang et al., 2024, lem. 2.3)

Let (s, U, t) be a weak h -hop negative sandwich and let (s, t) have $(r + h)$ -betweenness n/r . Then one can compute valid potentials φ making U r -remote in $O((r + h)(m + n \log n))$ time.

Betweenness + Sandwich $\rightarrow$ Remoteness

Proof.

- Define $\varphi(v) := \max\{d^{r+h}(s, v), -d^{r+h}(v, t)\}, v \in V$.

Betweenness + Sandwich $\rightarrow$ Remoteness

Proof.

- Define $\varphi(v) := \max\{d^{r+h}(s, v), -d^{r+h}(v, t)\}, v \in V$.
 $\forall u \in U$:
$$-d^{r+h}(u, t) \geq -d^h(u, t) \geq d^h(s, u) \geq d^{r+h}(s, u).$$
- Thus $\varphi(u) = -d^{r+h}(u, t) \geq d^h(s, u)$.

Betweenness + Sandwich $\rightarrow$ Remoteness

Proof.

- Define $\varphi(v) := \max\{d^{r+h}(s, v), -d^{r+h}(v, t)\}$, $v \in V$.
 $\forall u \in U$:
$$-d^{r+h}(u, t) \geq -d^h(u, t) \geq d^h(s, u) \geq d^{r+h}(s, u).$$
- Thus $\varphi(u) = -d^{r+h}(u, t) \geq d^h(s, u)$.
- For $v \in V$ with $d^{r+h}(s, v) + d^{r+h}(v, t) \geq 0$ we have
 $\varphi(v) = d^{r+h}(s, v)$.

Betweenness + Sandwich $\rightarrow$ Remoteness

Proof.

- Define $\varphi(v) := \max\{d^{r+h}(s, v), -d^{r+h}(v, t)\}, v \in V$.
 $\forall u \in U$:
 $-d^{r+h}(u, t) \geq -d^h(u, t) \geq d^h(s, u) \geq d^{r+h}(s, u)$.
- Thus $\varphi(u) = -d^{r+h}(u, t) \geq d^h(s, u)$.
- For $v \in V$ with $d^{r+h}(s, v) + d^{r+h}(v, t) \geq 0$ we have
 $\varphi(v) = d^{r+h}(s, v)$.
- For the same u, v , v is no longer reachable from u in r hops:

$$\begin{aligned}d_{\varphi}^r(u, v) &= d^r(u, v) + \varphi(u) - \varphi(v) \geq \\&d^r(u, v) + d^h(s, u) - d^{r+h}(s, v) \geq 0,\end{aligned}$$

by the triangle inequality.

Betweenness + Sandwich $\rightarrow$ Remoteness

Proof (cont.).

- Thus a vertex v is negatively reachable from u only if $d^{r+h}(s, v) + d^{r+h}(v, t) < 0$.
- There are only n/r such vertices. Thus, U is r remote with the reweighed function.

Betweenness + Sandwich $\rightarrow$ Remoteness

Proof (cont.).

- Thus a vertex v is negatively reachable from u only if $d^{r+h}(s, v) + d^{r+h}(v, t) < 0$.
- There are only n/r such vertices. Thus, U is r remote with the reweighed function.
- We can calculate $d^{r+h}(s, u)$ with the standard method.
- To calculate $d^{r+h}(u, t)$ we reverse the graph in $O(n + m)$ time, and use the standard method with source t . $\square$

Proper hops

A sandwich that is hard to make small:

Definition 21 (Proper h -hop walk)

A proper h -hop walk as a walk with exactly h negative edges and where all negative vertices are distinct.

Definition 22 (Proper h -hop distance)

For vertices s, t , let $\hat{d}^h(s, t)$ denote the infimum length over all proper h -hop walks from s to t . We call $\hat{d}^h(s, t)$ the proper h -hop distance from s to t .

A good enough failure

Lemma 23 (Huang et al., 2024, lem. 3.1)

There is an $O(h(m + n \log n))$ time algorithm that, given a set of vertices S , returns either a negative cycle, a pair $s, t \in S$ with $\hat{d}^h(s, t) < 0$, or the distances $d(V, v)$ in $G_S \ \forall v \in V$.

A good enough failure

Lemma 23 (Huang et al., 2024, lem. 3.1)

There is an $O(h(m + n \log n))$ time algorithm that, given a set of vertices S , returns either a negative cycle, a pair $s, t \in S$ with $\hat{d}^h(s, t) < 0$, or the distances $d(V, v)$ in $G_S \ \forall v \in V$.

Proof.

- Compute $\hat{d}^{h-1}(S, t)$ and $d^h(S, t)$ with the standard method in G_S
- If $d^h(S, t) = \hat{d}^{h-1}(S, t)$ then from **prelim** $d(S, t) = d^h(S, t)$.
- $d(V, t) = \min\{0, d(S, t)\}$

A good enough failure

Proof (cont.).

- Else $\exists t \in V : d^h(S, t) < d^{h-1}(S, t) \leq 0$.
- The walk from an $s \in S$ to t in h hops has precisely h negative edges
- If some negative vertex repeats, the walk between occurrences is negative. If it were not, we could remove it, and receive a $h - 1$ hop walk smaller than $d^h(S, t)$, which is a contradiction. Return the negative cycle.

A good enough failure

Proof(cont.).

- Else $\exists t \in V : d^h(S, t) < d^{h-1}(S, t) \leq 0$.
- The walk from an $s \in S$ to t in h hops has precisely h negative edges
- If some negative vertex repeats, the walk between occurrences is negative. If it were not, we could remove it, and receive a $h - 1$ hop walk smaller than $d^h(S, t)$, which is a contradiction. Return the negative cycle.
- Else, the vertices are distinct, and the walk is proper, as desired. □

Sampling Sandwiches

Large sandwiches can be sampled efficiently:

Sampling Sandwiches

Large sandwiches can be sampled efficiently:

Theorem 24 (Huang et al., 2024, lem. 4.1)

For $h = \Omega(\log n)$ there is a randomized algorithm that, in $O(h(m + n \log n))$ randomized time, returns either a negative cycle, a weak h -hop sandwich $(s, \mathcal{U}, t)$, or a set of negative vertices S and the distances $d(V, v)$, $v \in G_S$. With high probability, we have $|\mathcal{U}|, |S| \geq \Omega(\sqrt{hk})$.

Intermission: Chernoff Bound

We will use the following special case of the Chernoff bound:

Theorem 25 (Chernoff Bounds for binomials)

Let $X \sim \text{Bin}(n, p)$. Then $\mu = \mathbb{E}[X] = np$ and

$$\mathbb{P}\left[X \leq \frac{\mu}{2}\right] \leq e^{-\frac{\mu}{8}}$$

$$\mathbb{P}\left[X > \frac{3\mu}{2}\right] \leq e^{-\frac{\mu}{12}}$$

Sampling Sandwiches

- Randomly sample S from N with probability $\sqrt{\frac{h}{4k}}$ for each $v \in N$

Sampling Sandwiches

- Randomly sample S from N with probability $\sqrt{\frac{h}{4k}}$ for each $v \in N$
- $\mathbb{P} \left(|S| > \frac{1}{2}k\sqrt{\frac{h}{4k}} \right) \geq 1 - e^{-\frac{\sqrt{kh}}{32}} \quad \left(|S| \sim \text{Bin} \left(k, \sqrt{\frac{h}{4k}} \right) \right)$

Sampling Sandwiches

- Randomly sample S from N with probability $\sqrt{\frac{h}{4k}}$ for each $v \in N$
- $\mathbb{P} \left(|S| > \frac{1}{2}k\sqrt{\frac{h}{4k}} \right) \geq 1 - e^{-\frac{\sqrt{kh}}{32}} \quad \left(|S| \sim \text{Bin} \left(k, \sqrt{\frac{h}{4k}} \right) \right)$
- Now apply **lem. 3.1**, which may return:
 - (a) Negative Cycle
 - (b) The distances $d(V, t)$, $\forall t \in G_S$
 - (c) A pair (s, t) with $\hat{d}^h(s, t) < 0$. In this case, return $B_{s,t} := \{u \in N : d^h(s, u) + d^h(u, t) \leq 0\}$ in $O(h(m + n \log n))$ time.

Sampling Sandwiches

- Randomly sample S from N with probability $\sqrt{\frac{h}{4k}}$ for each $v \in N$
- $\mathbb{P}\left(|S| > \frac{1}{2}k\sqrt{\frac{h}{4k}}\right) \geq 1 - e^{-\frac{\sqrt{kh}}{32}} \quad \left(|S| \sim \text{Bin}\left(k, \sqrt{\frac{h}{4k}}\right)\right)$
- Now apply **lem. 3.1**, which may return:
 - (a) Negative Cycle
 - (b) The distances $d(V, t)$, $\forall t \in G_S$
 - (c) A pair (s, t) with $\hat{d}^h(s, t) < 0$. In this case, return $B_{s,t} := \{u \in N : d^h(s, u) + d^h(u, t) \leq 0\}$ in $O(h(m + n \log n))$ time.

Call (s, t) weak if the sandwich is small, $|B_{s,t}| \leq \sqrt{kh}$, strong otherwise.

Sampling Sandwiches

W.h.p., no weak pairs are sampled in by S .

Sampling Sandwiches

W.h.p., no weak pairs are sampled in by S . Assume (s, t) is weak.

- Any proper h -walk from s to t with negative length must include at least $h - 1$ vertices from $B_{s,t} \setminus \{s, t\}$.

Sampling Sandwiches

W.h.p., no weak pairs are sampled in by S . Assume (s, t) is weak.

- Any proper h -walk from s to t with negative length must include at least $h - 1$ vertices from $B_{s,t} \setminus \{s, t\}$.
- $\mathbb{E}[|B_{s,t} \cap S|] = \sum_{u \in B_{s,t}} \mathbb{E}[\{\text{sampled } u \text{ in } S\}] \leq \sqrt{\frac{h}{4k}} \sqrt{kh} = \frac{h}{2} \quad |B_{s,t} \cap S| \sim \text{Bin}\left(|B_{s,t}|, \sqrt{\frac{h}{4k}}\right)$

Sampling Sandwiches

W.h.p., no weak pairs are sampled in by S . Assume (s, t) is weak.

- Any proper h -walk from s to t with negative length must include at least $h - 1$ vertices from $B_{s,t} \setminus \{s, t\}$.
- $\mathbb{E}[|B_{s,t} \cap S|] = \sum_{u \in B_{s,t}} \mathbb{E}[\{\text{sampled } u \text{ in } S\}] \leq \sqrt{\frac{h}{4k}} \sqrt{kh} = \frac{h}{2}$
 $|B_{s,t} \cap S| \sim \text{Bin}\left(|B_{s,t}|, \sqrt{\frac{h}{4k}}\right)$
- Thus in order for $\hat{d}^h(s, t) < 0$, $|B_{s,t} \cap S|$ must be much further away from its expected value, which only happens with low probability (Chernoff).

Sampling Sandwiches

W.h.p., no weak pairs are sampled in by S . Assume (s, t) is weak.

- Any proper h -walk from s to t with negative length must include at least $h - 1$ vertices from $B_{s,t} \setminus \{s, t\}$.
- $\mathbb{E}[|B_{s,t} \cap S|] = \sum_{u \in B_{s,t}} \mathbb{E}[\text{sampled } u \text{ in } S] \leq \sqrt{\frac{h}{4k}} \sqrt{kh} = \frac{h}{2}$ $|B_{s,t} \cap S| \sim \text{Bin}\left(|B_{s,t}|, \sqrt{\frac{h}{4k}}\right)$
- Thus in order for $\hat{d}^h(s, t) < 0$, $|B_{s,t} \cap S|$ must be much further away from its expected value, which only happens with low probability (Chernoff).
- By the Union Bound:

$$\mathbb{P}[\{S \text{ is small}\} \cup \{(s, t) \text{ is weak}\}] \leq e^{-\frac{\sqrt{kh}}{4}} + e^{-\frac{\sqrt{kh}}{64}}$$

The Algorithm

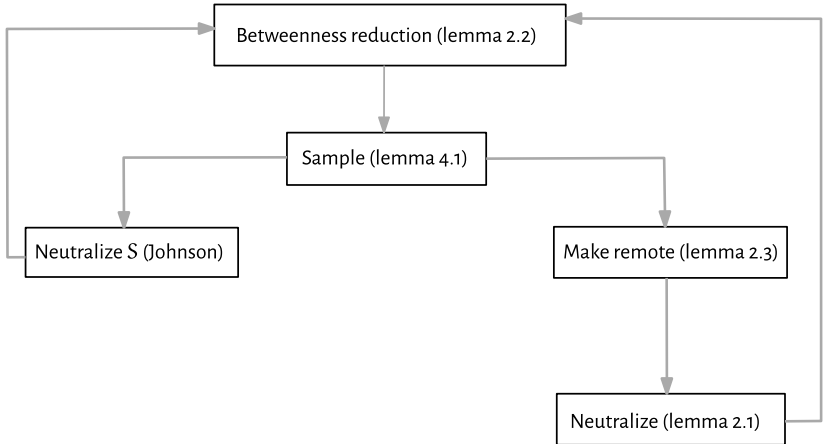


Figure 7: Flowchart illustrating the step-by-step execution of our algorithm.

The Algorithm

Theorem 26 (Huang et al., 2024, thm. 1.1)

There is a Las Vegas randomized algorithm solving the SSSP problem for real-weighted graphs that runs in

$$O\left(mn^{4/5} \log^{2/5} n + n^{9/5} \log^{7/5} n\right)$$

randomized time w.h.p.

The Algorithm

Proof.

While negative edges remain, repeat the following:

The Algorithm

Proof.

While negative edges remain, repeat the following:

- Assume $k = \Omega(\log^7(n))$, and set $h = \frac{k^{1/5}}{\log^{2/5} n} = \Omega(\log n)$
(otherwise run Dijkstra+BF)

The Algorithm

Proof.

While negative edges remain, repeat the following:

- Assume $k = \Omega(\log^7(n))$, and set $h = \frac{k^{1/5}}{\log^{2/5} n} = \Omega(\log n)$ (otherwise run Dijkstra+BF)
- By **lem. 2.2**, in $O(h^2(m \log n + n \log^2 n))$ time, we find a negative cycle, or compute a valid potential φ_1 to reduce the h betweenness of all pairs to n/h w.h.p.

The Algorithm

Proof.

While negative edges remain, repeat the following:

- Assume $k = \Omega(\log^7(n))$, and set $h = \frac{k^{1/5}}{\log^{2/5} n} = \Omega(\log n)$ (otherwise run Dijkstra+BF)
- By **lem. 2.2**, in $O(h^2(m \log n + n \log^2 n))$ time, we find a negative cycle, or compute a valid potential φ_1 to reduce the h betweenness of all pairs to n/h w.h.p.
- By **lem. 4.1**, we either find a negative cycle or S and the relevant Johnson potential φ_2 neutralizing S .

The Algorithm

Proof.

While negative edges remain, repeat the following:

- Assume $k = \Omega(\log^7(n))$, and set $h = \frac{k^{1/5}}{\log^{2/5} n} = \Omega(\log n)$ (otherwise run Dijkstra+BF)
- By **lem. 2.2**, in $O(h^2(m \log n + n \log^2 n))$ time, we find a negative cycle, or compute a valid potential φ_1 to reduce the h betweenness of all pairs to n/h w.h.p.
- By **lem. 4.1**, we either find a negative cycle or S and the relevant Johnson potential φ_2 neutralizing S .
- Else, we obtain a weak h -hop negative sandwich (s, U, t) .

The Algorithm

Proof.

While negative edges remain, repeat the following:

- Assume $k = \Omega(\log^7(n))$, and set $h = \frac{k^{1/5}}{\log^{2/5} n} = \Omega(\log n)$ (otherwise run Dijkstra+BF)
- By **lem. 2.2**, in $O(h^2(m \log n + n \log^2 n))$ time, we find a negative cycle, or compute a valid potential φ_1 to reduce the h betweenness of all pairs to n/h w.h.p.
- By **lem. 4.1**, we either find a negative cycle or S and the relevant Johnson potential φ_2 neutralizing S .
- Else, we obtain a weak h -hop negative sandwich (s, U, t) .
- If the reduction succeeded then, using **lem. 2.3** we can compute a potential φ_2 making U h -remote in $O(h(m + n \log n))$ time.

The Algorithm

Proof (cont.).

- Test if $\mathcal{U}$ is now remote in the same time, to check if the reduction succeed. If it failed, restart the loop.

The Algorithm

Proof (cont.).

- Test if $\mathcal{U}$ is now remote in the same time, to check if the reduction succeed. If it failed, restart the loop.
- By **lem. 2.1** find a negative cycle, or compute potentials φ_3 neutralizing $\mathcal{U}$ in $O((h + |\mathcal{U}|/h)(m + n \log n))$ time.

The Algorithm

Proof (cont.).

- Test if $\mathcal{U}$ is now remote in the same time, to check if the reduction succeed. If it failed, restart the loop.
- By **lem. 2.1** find a negative cycle, or compute potentials φ_3 neutralizing $\mathcal{U}$ in $O((h + |\mathcal{U}|/h)(m + n \log n))$ time.

In the worst case w.h.p., we neutralize

$$\ell = \Omega\left(\sqrt{kh}\right) = \Omega\left(\frac{k^{3/5}}{\log^{1/5} n}\right)$$

edges, or find a negative cycle at each iteration.

The Algorithm

Proof (cont. 2).

- Assuming $n = k$ negative vertices, halving k takes $O\left(k^{2/5} \log^{1/5} n\right)$ iterations. As the reduction is geometric, complexity is dominated by the halving of k .

The Algorithm

Proof (cont. 2).

- Assuming $n = k$ negative vertices, halving k takes $O\left(k^{2/5} \log^{1/5} n\right)$ iterations. As the reduction is geometric, complexity is dominated by the halving of k .
- The per step complexity is dominated by the reduction and the neutralization, with time $O\left(\left(h^2 + \frac{\ell}{h}\right) \left(m \log n + n \log^2 n\right)\right)$. Assuming $k \approx n$, $\ell \approx \sqrt{\frac{n}{h}}$, making $\ell \approx k^{2/5} \log^{1/5} n$.

The Algorithm

Proof (cont. 2).

- Assuming $n = k$ negative vertices, halving k takes $O\left(k^{2/5} \log^{1/5} n\right)$ iterations. As the reduction is geometric, complexity is dominated by the halving of k .
- The per step complexity is dominated by the reduction and the neutralization, with time $O\left(\left(h^2 + \frac{\ell}{h}\right) \left(m \log n + n \log^2 n\right)\right)$. Assuming $k \approx n$, $\ell \approx \sqrt{\frac{n}{h}}$, making $\ell \approx k^{2/5} \log^{1/5} n$.
- In total, it takes probabilistic time:

The Algorithm

Proof (cont. 3).

$$\begin{aligned} & O \left(\left(n^{2/5} \log^{1/5} n \right) \left(\frac{n^{2/5}}{\log^{4/5} n} + \sqrt{\frac{n}{h}} \right) (m \log n + n \log n) \right) = \\ & O \left(n^{4/5} \log^{2/5} n (m + n \log n) \right) = \\ & O \left(mn^{4/5} \log^{2/5} n + n^{9/5} \log^{7/5} n \right). \end{aligned}$$



Questions?

Bibliography

- [Bel58] Richard Bellman. **On a Routing Problem**. In: *Quarterly of Applied Mathematics* 16.1 (1958), pp. 87–90. ISSN: 0033-569X, 1552-4485. DOI: 10.1090/qam/102435. URL: <https://www.ams.org/qam/1958-16-01/S0033-569X-1958-0102435-2/>.
- [DI17] Yefim Dinitz and Rotem Itzhak. **Hybrid Bellman–Ford–Dijkstra Algorithm**. In: *Journal of Discrete Algorithms* 42 (Jan. 1, 2017), pp. 35–44. ISSN: 1570-8667. DOI: 10.1016/j.jda.2017.01.001. URL: <https://www.sciencedirect.com/science/article/pii/S1570866717300011>.
- [Dij59] Edsger W. Dijkstra. **A Note on Two Problems in Connexion with Graphs**. In: *Numerische Mathematik* 1 (1959), pp. 269–271.

- [Fin24] Jeremy T. Fineman. **Single-Source Shortest Paths with Negative Real Weights in $\tilde{O}(Mn^{8/9})$ Time**. In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*. STOC '24: 56th Annual ACM Symposium on Theory of Computing. Vancouver BC Canada: ACM, June 10, 2024, pp. 3–14. ISBN: 979-8-4007-0383-6. DOI: 10.1145/3618260.3649614. URL: <https://dl.acm.org/doi/10.1145/3618260.3649614>.
- [H]Q24] Yufan Huang, Peter Jin, and Kent Quanrud. **Faster Single-Source Shortest Paths with Negative Real Weights via Proper Hop Distance**. Dec. 9, 2024. DOI: 10.48550/arXiv.2407.04872. arXiv: 2407.04872 [cs]. URL: <http://arxiv.org/abs/2407.04872>. Pre-published.
- [Joh77] Donald B. Johnson. **Efficient Algorithms for Shortest Paths in Sparse Networks**. In: *Journal of the ACM* 24.1 (Jan. 1977), pp. 1–13. ISSN: 0004-5411, 1557-735X. DOI: 10.1145/321992.321993. URL: <https://dl.acm.org/doi/10.1145/321992.321993>.

Thank you!